

```

In [ ]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Load the dataset
iplData = pd.read_csv('D:/AI_Learning/LearningMaterial/EDA/IPL_Dataset.csv')

print("Dataset Overview (first few rows to understand the data):")
print(iplData.head())
print("\nDataset Shape:", iplData.shape)

# Q1. What is the maximum number of matches played by an individual player in a
maxMatchesPlayer = iplData.loc[iplData['Matches'].idxmax()]
print("\n1. Player with maximum matches in a season:")
print(f"{maxMatchesPlayer['Player']} - {maxMatchesPlayer['Matches']} matches")

# Q2. Top 2 players with maximum Average who have scored atleast 2 half centurie
playersHaving50s = iplData[iplData['50'] >= 2]
topAveragePlayers = playersHaving50s.sort_values(by='Avg', ascending=False).head()
print("\n2. Top 2 players with maximum average who scored at least 2 half centur
print(topAveragePlayers[['Player', 'Avg', '50']])

# Q3. Create 2 new columns based on Player name. First column will have first na
# Eg: for the player Shikhar Dhawan, Shikhar will be the first name and Dhawan w
iplData[['First_Name', 'Last_Name']] = iplData['Player'].str.split(' ', n=1, exp
iplData['Last_Name'] = iplData['Last_Name'].fillna('')
print("\n3. Split player names into first and last name:")
print(iplData[['Player', 'First_Name', 'Last_Name']].head())

# Q4. Create a new column (Cleaned_Highest_score) based on Highest score variabl
iplData['Cleaned_Highest_score'] = iplData['Highest Score'].str.replace('*', '')
print("\n4. Cleaned Highest Score column:")
print(iplData[['Player', 'Highest Score', 'Cleaned_Highest_score']].head())

# Q5. Print the total number of centuries scored in the entire season.
total_centuries = iplData['100'].sum()
print(f"\n5. Total number of centuries scored in the entire season: {total_centu

# Q6. Print all the player names whose strike rate is less than the average stri
# Print the player name, his strike rate and average strike rate
avgStrikeRate = iplData['Strike rate'].mean()
lowStrikeRatePlayers = iplData[iplData['Strike rate'] < avgStrikeRate]
print(f"\n6. (i) Average strike rate of all players: {avgStrikeRate:.2f}")
print("\n6. (ii) Player names whose strike rate is less than the average strike
for _, player in lowStrikeRatePlayers.head(10).iterrows(): # Showing first 10 f
    print(f"{player['Player']}: Strike Rate = {player['Strike rate']:.2f}")
print(f"\n6. (iii) Total players with below average strike rate: {len(lowStrikeR

# Q7. Please check the correlation between the features and create a heat map.
numericColumns = iplData.select_dtypes(include=['int64', 'float64']).columns
correlation = iplData[numericColumns].corr()
print("\n7. Correlation between the features:")
plt.figure(figsize=(12, 10))
sns.heatmap(correlation, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap')
plt.figure(figsize=(12, 8))
plt.tight_layout()

```

```

plt.savefig('correlation_heatmap.png')
plt.show()
print("\nHighest correlations:")
# Find top 5 highest absolute correlations (excluding self-correlations)
corrUnstack = correlation.unstack()
corrUnstack = corrUnstack[corrUnstack < 1.0] # Remove self-correlations
topCorr = corrUnstack.abs().sort_values(ascending=False).head(5)
for (col1, col2), value in topCorr.items():
    print(f"{col1} & {col2}: {correlation.loc[col1, col2]:.3f}")

# Q8. Check the List of players who has an average greater than 50 as well strik
highPerformers = iplData[(iplData['Avg'] > 50) & (iplData['Strike rate'] > 120)]
print("\n8. List of players who has an average greater than 50 as well strike ra
print(highPerformers[['Player', 'Avg', 'Strike rate']])

# Q9. Please check the List of players who has an average greater than 40 and ba
consistentPlayers = iplData[(iplData['Avg'] > 40) & (iplData['Balls faced'] > 10
print("\n9. List of players who has an average greater than 40 and balls faced a
print(consistentPlayers[['Player', 'Avg', 'Balls faced']])

# Q10. Players who scored atleast one century in this season. Create visualizati
centuryPlayers = iplData[iplData['100'] >= 1]
print("\nQ10. Players who scored at least one century:")
print(centuryPlayers[['Player', '100']])
plt.figure(figsize=(10, 6))
plt.bar(centuryPlayers['Player'], centuryPlayers['100'], color='orange')
plt.title('Players with At Least One Century')
plt.xlabel('Player')
plt.ylabel('Number of Centuries')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.figure(figsize=(12, 8))
plt.savefig('century_players.png')
plt.show()

# Q11. Players who scored atleast 4 half centuries in this season.
halfCenturyPlayers = iplData[iplData['50'] >= 4]
print("\n11. Players who scored at least 4 half centuries in this season:")
print(halfCenturyPlayers[['Player', '50']])

# Q12. Check the List of players who hit more than 45 boundaries and more than 1
boundaryHit45 = iplData[(iplData['4s'] > 45) & (iplData['6s'] > 10)]
print("\n12. List of players who hit more than 45 boundaries and more than 10 si
print(boundaryHit45[['Player', '4s', '6s']])

# Q13. Plot a histogram of number of matches played in a season by players.
plt.figure(figsize=(12, 8))
plt.hist(iplData['Matches'], bins=15, color='skyblue', edgecolor='black')
print("\n13. histogram of number of matches played in a season by players:")
plt.title('Distribution of Matches Played by Players')
plt.xlabel('Number of Matches')
plt.ylabel('Number of Players')
plt.grid(axis='y', alpha=0.75)
plt.savefig('matches_histogram.png')
plt.show()
print(f"Match count statistics: Min={iplData['Matches'].min()}, Max={iplData['Ma

# Q14. Plot the histogram of balls faced by players.
plt.figure(figsize=(12, 8))
plt.hist(iplData['Balls faced'], bins=20, color='lightgreen', edgecolor='black')

```

```

print("\n14. Histogram of balls faced by players:")
plt.title('Distribution of Balls Faced by Players')
plt.xlabel('Number of Balls Faced')
plt.ylabel('Number of Players')
plt.grid(axis='y', alpha=0.75)
plt.savefig('balls_faced_histogram.png')
plt.show()
print(f"Balls faced statistics: Min={iplData['Balls faced'].min()}, Max={iplData['Balls faced'].max()}")

# Q15. Top 10 players with most runs in a season.
top10RunScorers = iplData.sort_values(by='Runs', ascending=False).head(10)
print("\n15. Top 10 players with most runs in a season:")
print(top10RunScorers[['Player', 'Runs']])
plt.figure(figsize=(12, 6))
plt.bar(top10RunScorers['Player'], top10RunScorers['Runs'], color='green')
plt.title('Top 10 Run Scorers in IPL')
plt.xlabel('Player')
plt.ylabel('Runs')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.savefig('top_run_scorers.png')
plt.close()

# Q16. Print the players who played the match but didn't get the batting.
noBatting = iplData[iplData['Inns'] == 0] if len(iplData[iplData['Inns'] == 0]) > 0 else None
print("\n16. Players who played the match but didn't get the batting:")
print(noBatting[['Player', 'Matches', 'Inns', 'Runs']])

# Q17. Create a new column to show the percentage of total runs scored in 4s and 6s.
iplData['Boundary_Runs'] = (iplData['4s'] * 4) + (iplData['6s'] * 6)
iplData['Boundary_Run_Percentage'] = (iplData['Boundary_Runs'] / iplData['Runs']) * 100
iplData['Boundary_Run_Percentage'] = iplData['Boundary_Run_Percentage'].fillna(0)
boundaryPercentage = iplData[iplData['Runs'] >= 50].sort_values(by='Boundary_Run_Percentage', ascending=False)
print("\n17. Top 5 players with highest percentage of runs from boundaries:")
print(boundaryPercentage[['Player', 'Runs', 'Boundary_Runs', 'Boundary_Run_Percentage']])

# Q18. Print the players with top 5 Not out percentages (Not Out percentage can be calculated as (Not Out / Total Innings) * 100)
iplData['Not_Out_Percentage'] = (iplData['Not Out'] / iplData['Inns']) * 100
top5NotOutPercentage = iplData[iplData['Inns'] >= 3].sort_values(by='Not_Out_Percentage', ascending=False)
print("\n18. Top 5 players with highest not out percentage (min 3 innings):")
print(top5NotOutPercentage[['Player', 'Inns', 'Not Out', 'Not_Out_Percentage']])

# Q19. Create visualization of top 10 players with highest number of sixes.
top10SixHitters = iplData.sort_values(by='6s', ascending=False).head(10)
print("\n19. Top 10 players with most sixes:")
print(top10SixHitters[['Player', '6s']])
plt.figure(figsize=(12, 6))
plt.bar(top10SixHitters['Player'], top10SixHitters['6s'], color='purple')
plt.title('Top 10 Six Hitters in IPL')
plt.xlabel('Player')
plt.ylabel('Number of Sixes')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.savefig('top_six_hitters.png')
plt.show()

# Q20. Scatter plot of runs scored by a player v/s balls faced in a season. Then
plt.figure(figsize=(12, 6))
plt.scatter(iplData['Balls faced'], iplData['Runs'], alpha=0.7)
print("\n20. Scatter plot of runs scored by a player v/s balls faced in a season")

```

```
plt.title('Runs Scored vs Balls Faced')
plt.xlabel('Balls Faced')
plt.ylabel('Runs Scored')
plt.grid(True, alpha=0.3)

# Add regression line
z = np.polyfit(iplData['Balls faced'], iplData['Runs'], 1)
p = np.poly1d(z)
plt.plot(iplData['Balls faced'], p(iplData['Balls faced']), "r--")

# Calculate correlation
correlation = iplData['Balls faced'].corr(iplData['Runs'])
plt.annotate(f'Correlation: {correlation:.3f}', xy=(0.05, 0.95), xycoords='axes')
plt.tight_layout()
plt.savefig('runs_vs_balls.png')
plt.show()

print(f"\nCorrelation between Runs and Balls Faced: {correlation:.3f}")
print(f"Linear equation: Runs = {z[0]:.2f} * Balls_faced + {z[1]:.2f}")
```